

Polya Method in Computational Design Thinking

Detailed lecture PPT for B.Tech
students

1. What is Polya Method?

Polya's Method is a systematic approach to solving problems.

It was developed by mathematician George Polya and presented in How to Solve It.

It has four stages: Understand the Problem → Devise a Plan → Carry Out the Plan → Look Back.

In Computational Design Thinking, these stages help convert a problem into a computational solution.

2. Why Use Polya Method?

Students often start coding before clearly understanding the problem.

Polya gives a structured way to think before implementation.

It separates understanding, planning, implementation, and checking.

It encourages systematic thinking instead of random trial-and-error.

3. Four Steps at a Glance

1. Understand the Problem — identify requirements, inputs, outputs, and constraints.
2. Devise a Plan — decide the approach and break the problem into manageable parts.
3. Carry Out the Plan — implement the planned algorithm or computational solution.
4. Look Back — test, verify, evaluate, and improve the solution.

4. Step 1 — Understand the Problem

First, clearly identify what needs to be solved.

INPUT: What information/data is available?

OUTPUT: What result is required?

REQUIREMENTS: What must the solution do?

CONSTRAINTS: What limitations or conditions must be followed?

Clarify assumptions and remove ambiguity before designing the solution.

5. Step 1 — Questions to Ask

What exactly is the problem?

What information is given?

What do we need to find or produce?

What are the constraints?

Who or what will use the solution?

Can the problem be stated more clearly or simply?

6. Step 1 — Connection to Computational Thinking

Understanding prepares us for abstraction and decomposition.

We identify relevant information and separate it from unnecessary details.

We recognize inputs, outputs, rules, conditions, and constraints.

A clearly defined problem supports the design of a computational model.

7. Step 2 — Devise a Plan

After understanding the problem, decide how it can be solved.

Select a suitable strategy or computational approach.

Break a complex problem into smaller sub-problems when necessary.

Think about algorithms, data, patterns, conditions, and computational models.

The plan should be clear enough to implement.

8. Step 2 — Questions to Ask

Have I seen a similar problem before?

Can I divide the problem into smaller parts?

What algorithm or strategy could work?

What data will be needed?

What computational model or representation is suitable?

What could go wrong with this approach?

9. Step 2 — From Plan to Algorithm

A plan becomes computational when it can be represented through computational steps.

Use pseudocode or a flowchart to express logic before coding.

Define operations, conditions, repetitions, and required data.

An algorithm is not merely any sequence of steps; it controls a computational model without requiring human judgment during execution.

10. Step 3 — Carry Out the Plan

Now implement the planned solution.

Convert the algorithm or pseudocode into executable instructions where appropriate.

Use programming, simulation, data processing, or another computational method.

Follow the planned logic carefully.

Record errors and unexpected results for later debugging.

11. Step 3 — Implementation Checklist

Are the planned steps being followed correctly?

Are inputs handled correctly?

Are conditions and loops working as intended?

Does the implementation produce the required output?

Are errors identified and corrected?

Can another person understand or reproduce the solution?

12. Step 4 — Look Back / Review

A solution is not complete just because it runs.

Check whether it satisfies the original requirements.

Test normal, boundary, and unusual cases where appropriate.

Look for errors, inefficiency, missing cases, and unnecessary complexity.

Improve the solution based on testing results.

13. Step 4 — Questions to Ask

Is the solution correct?

Does it solve the original problem?

Does it satisfy all constraints?

What happens for different inputs?

Are there edge cases?

Can the solution be simpler, faster, safer, or more reliable?

14. Polya + Computational Design Thinking

UNDERSTAND → problem definition, requirements, inputs/outputs, constraints.

PLAN → abstraction, decomposition, computational model, algorithm, pseudocode.

EXECUTE → implementation of the designed computational solution.

REVIEW → testing, evaluation, debugging, and improvement.

The process can be iterative: review may send us back to understanding or planning.

15. Complete Flow

PROBLEM



1. UNDERSTAND — What is the actual problem?



2. PLAN — How can we represent and solve it computationally?



3. EXECUTE — Implement the plan.



4. LOOK BACK — Test, evaluate, and improve.

↻ Revise an earlier step if testing reveals a problem.

16. Example — ATM Withdrawal System

Understand: Identify the amount, account information, PIN, balance, and withdrawal conditions.

Plan: Design the sequence and decision logic for authentication, balance checking, amount validation, and transaction processing.

Execute: Implement the designed algorithm in the ATM/software system.

Look Back: Test correct PIN, wrong PIN, insufficient balance, invalid amount, and other relevant cases.

17. Example — Student Result System

Understand: Given marks, calculate total/percentage and determine the result according to defined rules.

Plan: Identify inputs, calculations, conditions, and required output; create pseudocode/flowchart.

Execute: Implement the algorithm in a program.

Look Back: Test different marks, boundary values, invalid inputs, and verify the results.

18. Common Mistakes

Starting coding before understanding the problem.

Choosing an algorithm without checking requirements.

Ignoring constraints and edge cases.

Testing only one normal input.

Assuming 'program runs' means 'solution is correct'.

Not reviewing or improving the solution after testing.

19. Advantages of Polya Method

Provides a clear structure for problem solving.

Encourages students to think before implementation.

Connects problem understanding with algorithm design.

Makes testing and reflection an explicit part of the process.

Supports systematic debugging and improvement.

Can be applied to mathematical and computational problems.

20. Exam-Oriented Definition

Polya's Method is a four-step systematic problem-solving method.

1. Understand the Problem
2. Devise a Plan
3. Carry Out the Plan
4. Look Back / Review the Solution

In Computational Design Thinking, it guides the transition from problem understanding to planning, implementation, testing, and improvement.

21. Quick Revision

Understand = What is the problem?

Plan = How will I solve it?

Execute = Implement the solution.

Review = Does it work correctly and how can it be improved?

Remember: THINK → PLAN → DO → CHECK.

22. Key Takeaway

Do not directly jump from a problem statement to code.

First understand the problem.

Then design a suitable computational approach.

Implement it carefully.

Finally, test and improve it.

Polya's method fits naturally into systematic computational problem solving.